

EXPERIMENT-18

1. Problem Understanding:

The program displays a menu and performs one of four operations on a positive integer:

1. Strong Number — A number is Strong if the sum of the factorials of its digits equals the number.

Example: $145 = 1! + 4! + 5! = 145$

2. Armstrong Number — A number is Armstrong if the sum of each digit raised to the power of the number of digits equals the original number.

Example: $153 = 1^3 + 5^3 + 3^3 = 153$

3. Harshad Number — A number is Harshad if it is divisible by the sum of its digits.

Example: $18 \rightarrow 1 + 8 = 9$, and $18 \% 9 = 0$

4. Duck Number — A positive integer is a Duck number if it contains at least one 0 in its digits.

Example: 1023 is a Duck number.

2. Algorithm:

1. Start.

2. Declare integer variable choice.

3. Display the four menu options.

4. Read the user's choice.

5. Use switch(choice).

6. If choice is 1, call strongNumber().
7. If choice is 2, call armstrongNumber().
8. If choice is 3, call harshadNumber().
9. If choice is 4, call duckNumber().
10. For any other choice, display "Invalid choice".
11. Stop.

Function 1: Strong Number:

1. Read a positive integer n.
2. Store n in temp.
3. Initialize sum = 0.
4. Repeat while temp > 0:

Extract the last digit using $\text{digit} = \text{temp} \% 10$.

Find the factorial of digit.

Add the factorial to sum.

Remove the last digit using $\text{temp} = \text{temp} / 10$.

5. If sum == n, display that the number is a Strong number.
6. Otherwise, display that it is not a Strong number.

Function 2: Armstrong Number:

1. Read a positive integer n.
2. Store n in temp.
3. Count the number of digits in n.
4. Again store n in temp.

5. Initialize sum = 0.
6. Extract each digit.
7. Calculate digit raised to the power of the number of digits.
8. Add the result to sum.
9. Remove the processed digit.
10. If sum == n, display that it is an Armstrong number.
11. Otherwise, display that it is not an Armstrong number.

Function 3: Harshad Number:

1. Read a positive integer n.
2. Store n in temp.
3. Initialize sum = 0.
4. Extract each digit and add it to sum.
5. Remove each processed digit.
6. Check $n \% \text{sum}$.
7. If the remainder is 0, display that it is a Harshad number.
8. Otherwise, display that it is not a Harshad number.

Function 4: Duck Number:

1. Read a positive integer n.
2. Store n in temp.
3. Initialize isDuck = 0.
4. Extract the digits one by one.
5. If any digit is 0, set isDuck = 1.
6. If isDuck == 1, display that it is a Duck number.
7. Otherwise, display that it is not a Duck number.

3. Test Case Table:

Case	Choice	Input	Expected Output	Actual Output	Result
1	1	145	145 is a Strong number	145 is a Strong number	Pass
2	1	123	123 is not a Strong number	123 is not a Strong number	Pass
3	2	153	153 is an Armstrong number	153 is an Armstrong number	Pass
4	2	123	123 is not an Armstrong number	123 is not an Armstrong number	Pass
5	3	18	18 is a Harshad number	18 is a Harshad number	Pass
6	3	19	19 is not a Harshad number	19 is not a Harshad number	Pass
7	4	1023	1023 is a Duck number	1023 is a Duck number	Pass
8	4	1234	1234 is not a Duck number	1234 is not a Duck number	Pass

4. Program, Compile and Execute:

5. Conclusion:

The program satisfies the specified requirement because the calls are simply `strongNumber()`, `armstrongNumber()`, `harshadNumber()`, and `duckNumber()`—no arguments are passed, every function has a void return type, and each function takes its own required input.

GNU nano 9.0

```
#include <stdio.h>

/* Function declarations */
void strongNumber();
void armstrongNumber();
void harshadNumber();
void duckNumber();
int main()
{
    int choice;

    printf("1. Check Strong Number\n");
    printf("2. Check Armstrong Number\n");
    printf("3. Check Harshad Number\n");
    printf("4. Check Duck Number\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch(choice)
    {
        case 1:
            strongNumber();
            break;

        case 2:
            armstrongNumber();
            break;

        case 3:
            harshadNumber();
            break;

        case 4:
            duckNumber();
            break;

        default:
            printf("Invalid choice\n");
    }

    return 0;
}
```

GNU nano 9.0

```
/* Function to check Strong Number */
void strongNumber()
{
    int n, temp, digit;
    int sum = 0, fact, i;

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    temp = n;

    while(temp > 0)
    {
        digit = temp % 10;

        fact = 1;

        for(i = 1; i ≤ digit; i++)
        {
            fact = fact * i;
        }

        sum = sum + fact;
        temp = temp / 10;
    }

    if(sum == n)
        printf("%d is a Strong number.\n", n);
    else
        printf("%d is not a Strong number.\n", n);
}

/* Function to check Armstrong Number */
void armstrongNumber()
{
    int n, temp, digit;
    int digits = 0, sum = 0;
    int power, i;

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    temp = n;
```

GNU nano 9.0

```
/* Count number of digits */
while(temp > 0)
{
    digits++;
    temp = temp / 10;
}

temp = n;

while(temp > 0)
{
    digit = temp % 10;

    power = 1;

    for(i = 1; i ≤ digits; i++)
    {
        power = power * digit;
    }

    sum = sum + power;
    temp = temp / 10;
}

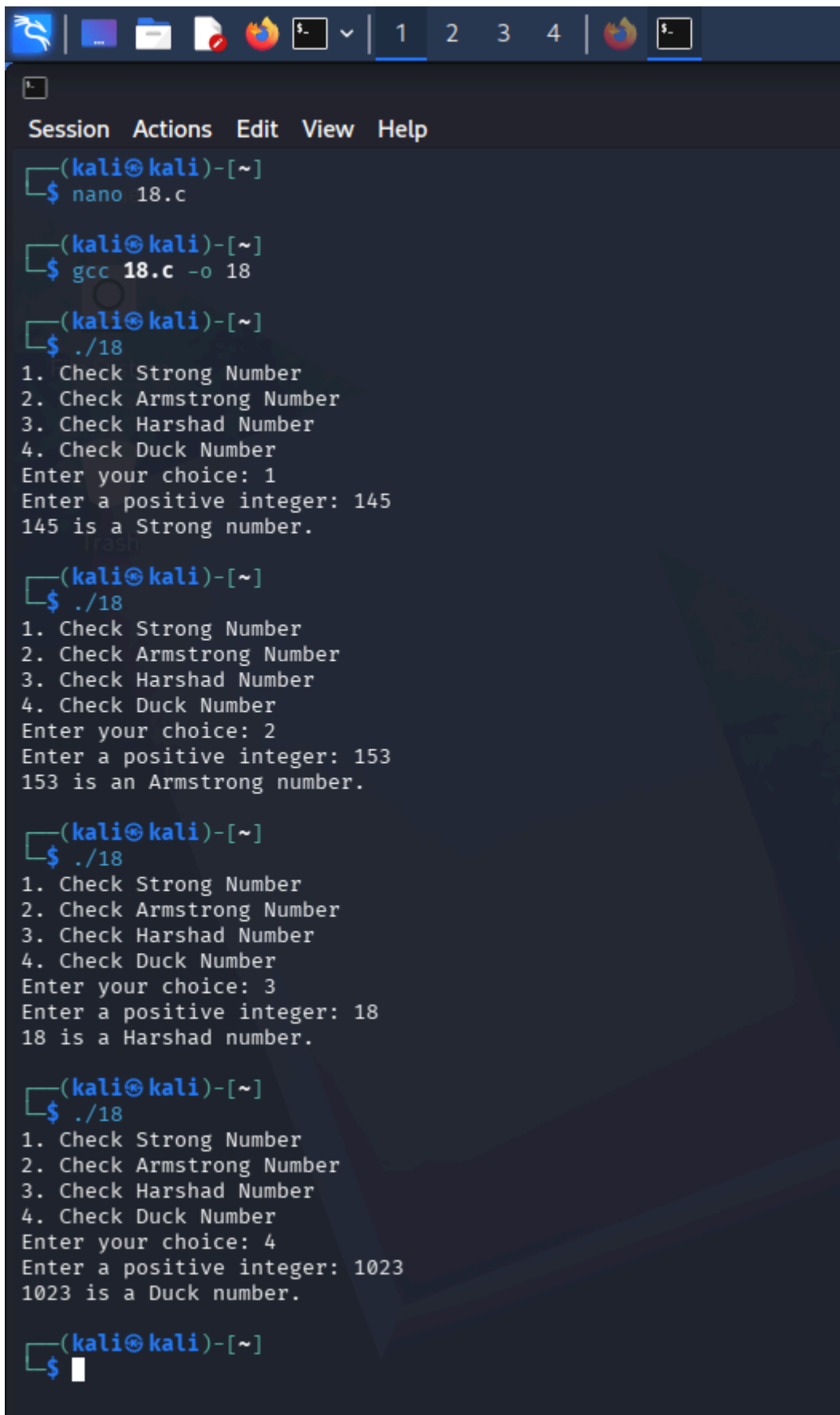
if(sum == n)
    printf("%d is an Armstrong number.\n", n);
else
    printf("%d is not an Armstrong number.\n", n);
}

/* Function to check Harshad Number */
void harshadNumber()
{
    int n, temp;
    int sum = 0;

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    temp = n;

    while(temp > 0)
    {
        sum = sum + (temp % 10);
        temp = temp / 10;
    }
}
```



```
(kali㉿kali)-[~]  
$ nano 18.c  
  
(kali㉿kali)-[~]  
$ gcc 18.c -o 18  
  
(kali㉿kali)-[~]  
$ ./18  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 1  
Enter a positive integer: 145  
145 is a Strong number.  
  
(kali㉿kali)-[~]  
$ ./18  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 2  
Enter a positive integer: 153  
153 is an Armstrong number.  
  
(kali㉿kali)-[~]  
$ ./18  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 3  
Enter a positive integer: 18  
18 is a Harshad number.  
  
(kali㉿kali)-[~]  
$ ./18  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 4  
Enter a positive integer: 1023  
1023 is a Duck number.  
  
(kali㉿kali)-[~]  
$
```


NAME: TAANISH AGARWAL
BATCH: B-15
SAP ID: 590034662